13 and 14, we try to load a model from the directory *existing_model*, if such a model is already trained and saved there. The first time you execute this code, no model is existing and hence an exception will be raised. Line 15 handles this exception by executing lines 16 to 21 which define, train and save our model. The 16th line of code (which spans over multiple lines) defines our model. The parameters given in this line decide the behaviour of our model. We are using a sequential model having a stack of layers such that each layer has exactly one input tensor and one output tensor. Our discussion of the parameters used to define the model is deferred till the next article in this series. Since neural networks are so important in machine learning, our discussion then will cover a lot of details. Until then, consider the neural network used here as a black box.

Line 17 defines the batch size as 64. This line of code decides the number of samples per batch of computation. Line 18 defines the number of epochs as 25. This value decides the number of times the entire data set will be processed by the learning algorithm. Line 19 configures the model for training. Line 20 trains the model for the given number of epochs, which in this case is 25. The detailed explanation of these two lines of code is also deferred till the next article. Finally, Line 21 saves our trained model to the directory named *existing_model*. You will find the model saved in this directory as a number of *.pb* (TensorFlow) files. Notice that lines 16 to 21 are inside the *except* block.

```
22. print(model.summary( ))
23. score = model.evaluate(x_test, y_
test, verbose=0)
24. print("Test loss:", score[0])
25. print("Test accuracy:", score[1])
```

Line 22 prints a summary about the model trained by us. Figure 6 shows this summary of the model trained.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 26, 26, 32)        320

 max_pooling2d (MaxPooling2D  (None, 13, 13, 32)       0
 )

 conv2d_1 (Conv2D)           (None, 11, 11, 64)        18496

 max_pooling2d_1 (MaxPooling  (None, 5, 5, 64)         0
 2D)

 flatten (Flatten)           (None, 1600)              0

 dropout (Dropout)           (None, 1600)              0

 dense (Dense)               (None, 10)                16010

=================================================================
Total params: 34,826
Trainable params: 34,826
Non-trainable params: 0
```

Figure 6: Details of the model trained

Recall that the data set loaded in the beginning was divided into training data and test data. Line 23 uses the test data to test the accuracy of the model we have trained. Lines 24 and 25 print the details of this test.

```
26. img = keras.utils.load_
img("sample1.png").resize((28, 28)).
convert('L')
27. img = keras.utils.img_to_array(img)
28. img = img.reshape((1, 28, 28, 1))
29. img = img.astype('float32')/255
30. score = model.predict(img)
31. print(score)
32. print("Number is",
np.argmax(score))
33. print("Accuracy", np.max(score) *
100.0)
```

Now, it is time for us to deploy our model. I have written a few digits on paper and scanned them so that our model can use these images for classification. Figure 7 shows one of the images I have used to test our model. Line 26 loads our image after converting it to grayscale and resizing it to 28 x 28 pixels. Lines 27 to 29 do the necessary pre-processing so that the image can be given as input to our trained model. Line 30 predicts the class to which the image belongs to. Lines 31 to 33 print the details of this prediction. Figure 8 shows this part of the output of the program *digit.py*. From the figure it can be seen that, though the image is correctly identified as 7, the accuracy is only 23.77%. Further, from the value of the list score shown in Figure 8, it can be verified that the same number was identified as 1 with 12.86% accuracy and as 8 or 9 with about 11% accuracy. Moreover, the model even failed to identify the digit correctly on a few occasions. Though I couldn't pinpoint any reasons for this sub-par performance, I think the relatively low resolution (28 x 28 pixel) training images as well as the quality of the scanned images I have generated could be the main contributing factors. Though not the best model available in town, we now have our first trained model based on AI and machine learning principles. Hopefully, in the coming articles in this series, we will build models